

交通大数据

机器学习简介

- 刘志远教授
- Email: zhiyuanl@seu.edu.cn



机器学习简介

□ 学习目标

- 了解机器学习的基础概念、主要任务和基本流程
- 掌握机器学习模型的评估与选择方法
- 掌握改善模型性能的方法与技巧



1 机器学习的基础概念

- 机器学习的定义与对象

- 常用概念

- 机器学习的分类



机器学习的定义与对象

我们已经学习了交通大数据分析的数据预处理、探索性数据分析以及图表绘制的过程。通过这些工作，我们已经能够将杂乱的数据梳理清晰，并在统计维度上对数据有了充分的认识。但是，面对复杂的大数据问题，我们更需要对这些数据进行深入的挖掘和探索，并在其基础上构建模型，对未知问题进行预测和分析。致力于这一目标的，就是机器学习（machine learning）。

机器学习研究如何通过计算手段，利用经验来改善系统自身的性能，而在计算机系统中，“经验”通常以“数据”的形式存在。因此，机器学习研究的主要内容，是关于从数据中产生“模型”的算法，即“学习算法”，并通过运用这些算法，对未知数据进行预测和分析。



机器学习的定义与对象

Herbert Simon 曾对“学习”给出定义：“如果一个系统能够通过执行某个过程来根据经验改进它的性能，这就是学习。”按照这一观点，机器学习的过程可以表示成下图所示的形式：

在数据基础上，首先选取合适的算法构建模型，并对性能进行评估，如果性能达到要求则应用该模型输出分类和预测的结果，若性能不理想，则调整算法或模型参数等建立新模型，再次评估和验证，如此循环往复，最终确定最优模型。

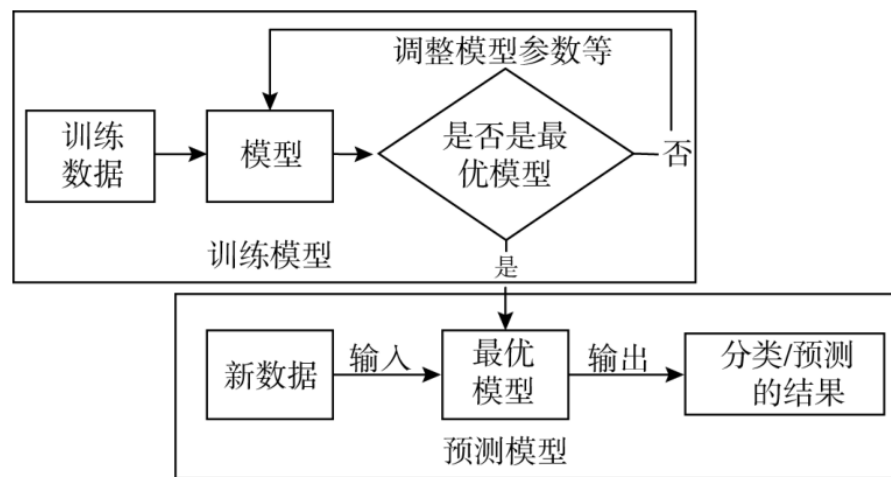


图 机器学习过程



机器学习的定义与对象

机器学习的关键在于：计算机能否从数据中自行学习经验并产生满足需求的模型，即是否能实现“学习”的行为。我们以一个具有多年驾驶经验的司机开车为例，来加深对于“学习”的理解。当他要在陌生道路行驶，虽然不熟悉路况，但由于他在各种路况下都驾驶过，积累了丰富的经验，因此面对陌生的道路也可以很快适应，这就是一个自我学习并应用到新场景中的过程



机器学习的定义与对象

机器学习的对象是数据。机器学习从原始数据出发，通过提取数据的特征组成结构化数据，抽象出结构化数据的模型，发现原始数据中的知识，最后将其应用到对新数据的分析与预测中。需要注意的是，这里的数据是广义且多样的，包括各类数字、文字、图形等等

为了合理利用数据，机器学习也需要对数据进行一定的基本假设，要求同类数据具有规律性或者内部结构。例如，通常假设我们获得的样本都是“独立同分布”的，即每个样本都是独立、无差别地从整体中进行抽样的；此时，每个样本服从的概率分布 D 与总体 (population) 的概率分布是相同的。



机器学习的常用概念

□ 样本与特征

机器学习所需数据实际上是对事件或研究对象的记录，每一条数据记录都可以被称作一个样本；样本包含的事件或研究对象的性质被称为特征。如一个城市道路的交通流数据样本会包含“路段编号”“平均车速”“车流量”等特征

□ 数据集

数据集是由样本组成的集合。在机器学习中数据集一般被手动划分为三类：**训练集**，**验证集**和**测试集**。三种数据集分别起到训练模型、选择模型和模型评估的作用

□ 特征向量与特征空间

以 $D = \{x_1, x_2, \dots, x_m\}$ 表示包含 m 个样本的数据集，每个样本由 d 个特征描述，任意一个样本 $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$ 是 d 维空间 χ 中的一个向量，所以 x_i 也被成为**特征向量**。 d 被称为样本维度（dimensionality）。样本在各个特征维度上的取值范围组成了 χ ， χ 被称为**特征空间**（也称为属性空间、样本空间、输入空间）

一条样本

路段编号	流量(pcu/h)	速度 (m/s)	车道宽度(m)	是否出现拥堵(0/1)
1	1200	36	3.75	1
2	1130	32	3.75	0
3	1580	27	3.5	1



机器学习的常用概念

□ 未标注数据与标注数据

含有标签的数据被称为**标注数据**，反之，不含有标签的数据被称为**未标注数据**。标签是对数据所描述状态的判断。例如，对于交通流数据，我们可以根据数据给定“畅通”或“拥堵”的判断。我们用 (x_i, y_i) 表示第 i 个样本，其中 $y_i \in Y$ 是样本 $x_i \in \chi$ 的标签。 Y 是所有标签的集合，也被称为“标签空间”“输出空间”

□ 模型

模型是从输入空间 χ 到输出空间 Y 的一个映射函数 f 。 $y = f(x)$ ，模型 f 也被称为一个“**假设**” (hypothesis) 。

□ 假设空间

所有模型的集合被称为**假设空间**，例如，我们以 $y = f(x) = \theta_0 + \theta_1 x$ 分析平均速度和车流密度之间的关系，那么 (θ_0, θ_1) 的所有可能取值就定义了假设空间。

□ 超参数

模型中的各个系数被称为参数，对于事先设定好的，在学习过程中保持不变的参数被称为“超参数”，例如线性回归模型中变量的个数、聚类中类的个数、神经网络的层数等。



机器学习的分类

根据问题类别和建模方式的差异，机器学习中的模型与算法主要分为以下四类：监督学习（supervised learning）、无监督学习（unsupervised learning）、半监督学习（semi-supervised learning）和强化学习（reinforcement learning）

分类	主要任务	特点
监督学习	分类；回归	有特征，有标签，即有标准答案
半监督学习	分类；回归；聚类	有特征，部分样本有标签
无监督学习	聚类；降维	有特征，无标签，即无标准答案
强化学习	决策	无标签，有状态，有决策过程和奖励机制，学习一系列行为使学习目标最大化



2 机器学习的主要任务

根据训练数据是否有标签，学习任务被大致划分为两类，监督学习和无监督学习。监督学习旨在分析输入变量和输出变量之间的关系，构建模型找出两者之间的变化规律，若输出变量是连续的，那么这类学习任务被称为回归（regression），如果输出变量为离散的，那么这一学习任务就是一个分类（classification）问题。

无监督学习通过未标注数据来揭示数据的内在规律和性质，无监督学习主要包括聚类（clustering）和降维（dimension reduction）问题

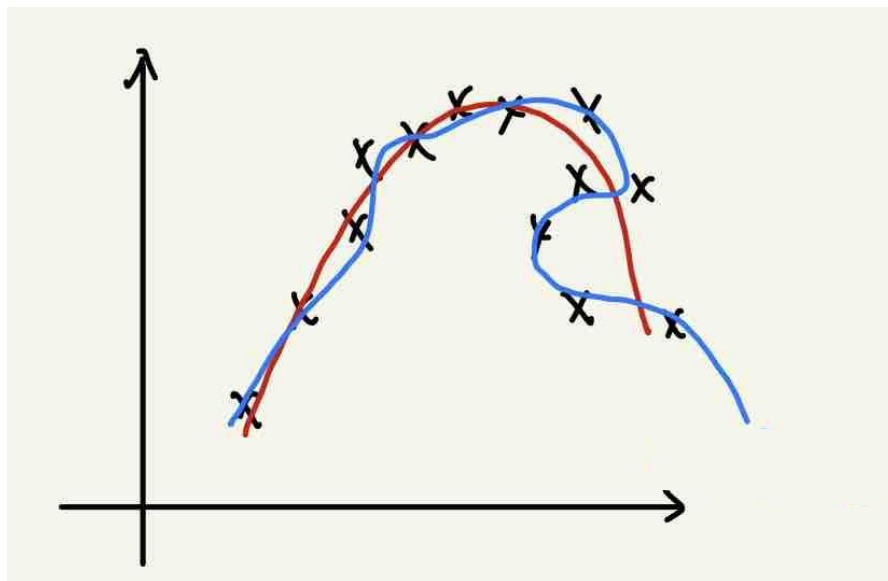
- 回归问题
- 分类问题
- 聚类问题
- 降维问题



□ 回归问题

回归问题旨在学习输入空间 $\mathcal{X}$ 到输出空间 $\mathcal{Y}$ 的映射函数 $f: \mathcal{X} \rightarrow \mathcal{Y}$ 。这个“学习”过程被称为“回归”或“拟合”，即选择一条直线或曲线 $y = f(x)$ 逼近已知数据，从而预测未知数据。按照输入变量的个数，回归问题可划分为一元回归和多元回归；而按照模型的类型，又可分为线性回归和非线性回归

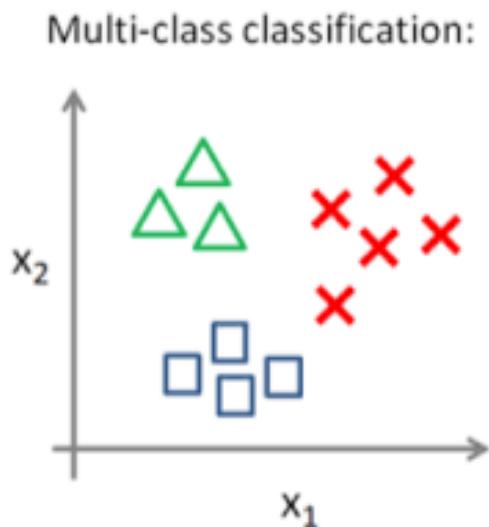
在食品领域，一些红酒厂商根据红酒的酸性、含糖量、氯化物含量、PH值、密度等信息，建立回归算法，对红酒品质进行把控；在交通领域，回归算法可以用于预测交通流量



□ 分类问题

分类问题旨在通过已有数据 (x, y) 来构建学习器，从而能对新的输入数据 x 精准地预测出其正确的分类 $\bar{y}$ 。这里的学习器也被称为分类模型、分类决策模型、分类器。

交通领域分类算法常用于识别交通拥堵状态，识别交通风险点等；在电商平台，分类算法被应用于电商优惠券发放、客户分类、广告推送等；在医学上，分类算法越来越多地被应用于肿瘤识别、自动化疾病监测等方面



回归问题 VS 分类问题

回归问题与分类问题本质区别在于哪里？我们能否将回归问题与分类问题相互转化？（例如回归方法结合阈值处理分类问题）

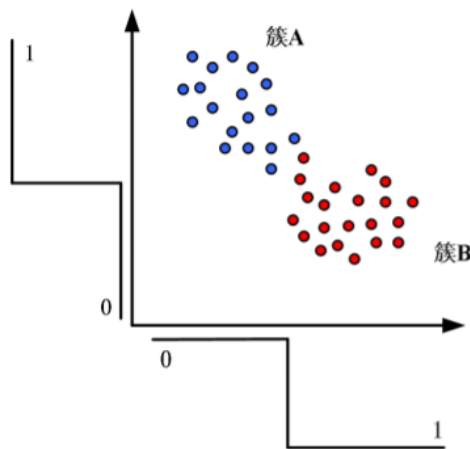


□ 聚类问题

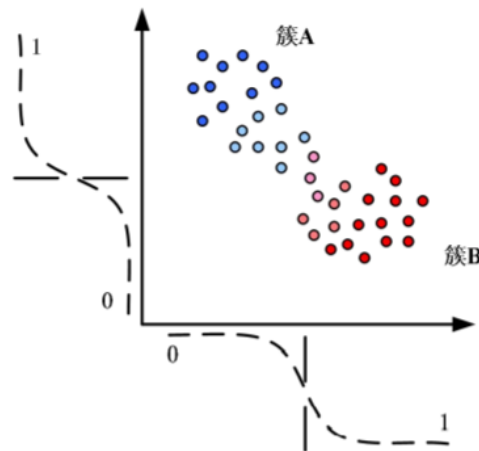
聚类是无监督学习中最典型的一类问题，它将样本集中相似的样本分配到同一类别中，从而得到了一些不同的类或簇。**聚类并不要求事先给定类别，而是让模型从数据中自动学习类别划分。**聚类问题可以被分为两类，硬聚类（hard clustering）和软聚类（soft clustering）

硬聚类：把每一个样本确切分到某一个聚类中

软聚类：允许每个样本可以属于多个类，并定义数据点与各个聚类的成员水平（0-1之间的概率值，可以显示数据点与某一聚类之间联系的强弱）



(a) 硬聚类



(b) 软聚类

聚类算法的一个典型应用是**新闻编辑**，新闻网站每天搜集上万条新闻（无标签），对新闻进行聚类，将有相关性的新闻放在同一板块中



时事要闻

相关话题: [中国财经](#) [国际财经](#) [深度分析](#)

美国参议院民主党领袖舒默指示议员起草法案以应对中国崛起

路透华盛顿2月23日 - 美国参议院民主党领袖舒默(Chuck Schumer)周二表示, 他已经指示议员们研拟一套措施以应对中国崛起, 利用国会两党对中国持强硬立场的氛围, 强化美国科技业和对抗不公平做法。 [全文](#)

独家: 美国参议员将恢复立法 对抗中国对美企的

路透华盛顿2月23日 - 美国参议员Jeff Merkley对路透表示, 国会这项新举措是要让中国为其不断努力压制国外批 [全文](#)

拜登与特鲁多在首次双边会谈中将搁置分歧 聚

路透华盛顿/渥太华2月23日 - 美国总统拜登和加拿大总理特而产生的分歧翻篇, 寻求在前总统特朗普的动荡岁月后重塑 [全文](#)

对于缅甸抗议者遭到暴力对待 七国集团外交部

图片新闻



货拉拉就女孩跳车身亡事件致歉

2月24日, 货拉拉就长沙女孩搬家途中跳车身亡道歉: 平台负有不可推卸的责任, 再次对当事人车女士的家属致以最诚挚的... [详细 >](#)

北京日报客户端 1小时前



事件脉络

- 2月24日 09:25 货拉拉就女孩跳车身亡事件致歉
- 2月23日 23:00 女孩坐货拉拉出事前一分钟监控曝光
- 2月23日 20:27 货拉拉司机家属回应
- 2月23日 19:20 女孩坐货拉拉跳车身亡 司机被刑拘
- 2月23日 12:08 警方成立专案组调查女孩跳车身亡
- 2月23日 09:39 医生讲述跳车女孩送医时伤情细节

资讯 >



最新进展! 货拉拉为女孩跳车身亡事件致歉 将上线强制...

2月24日, 货拉拉在微博发布关于用户跳车事件的致歉和整改公告。货拉拉表示, "对事件的发生, 平台负有不可推卸的责任"。货拉拉...

中金在线 10分钟前



货拉拉就长沙女生跳车身亡致歉 表示将整改

新浪科技讯 2月24日上午消息, 针对关于用户跳车事件, @货拉拉App 发布关于用户跳车事件的致歉和整改公告。货拉拉表示, 货拉拉方...

新浪科技 1小时前

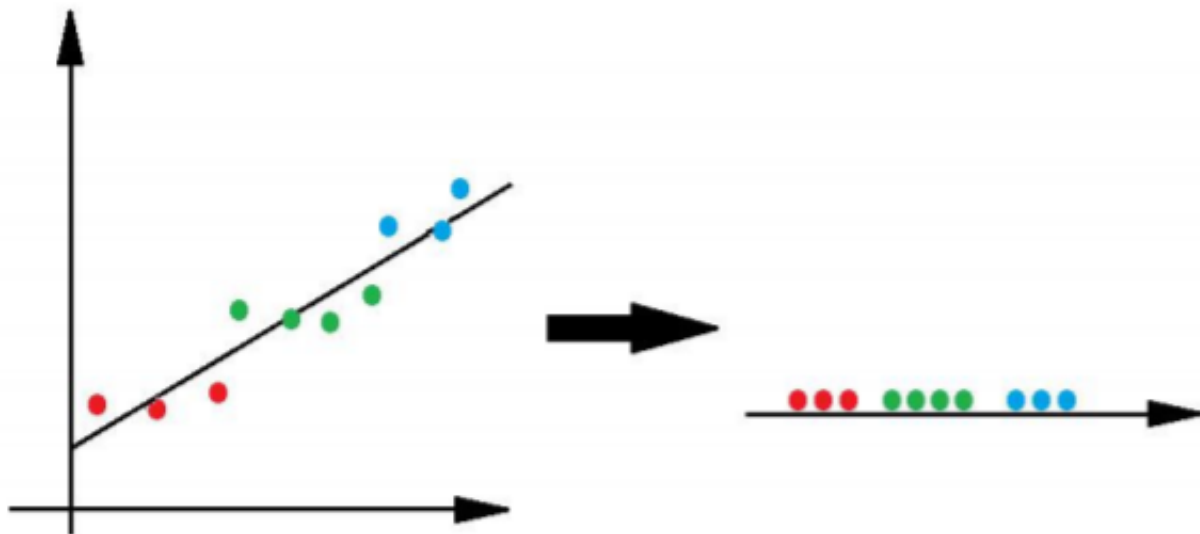


货拉拉为女孩跳车身亡事件致歉 承认有三个"明显问题..."

原标题: 长沙女孩搬家途中跳车身亡, 货拉拉发文承认平台有三个"明显问

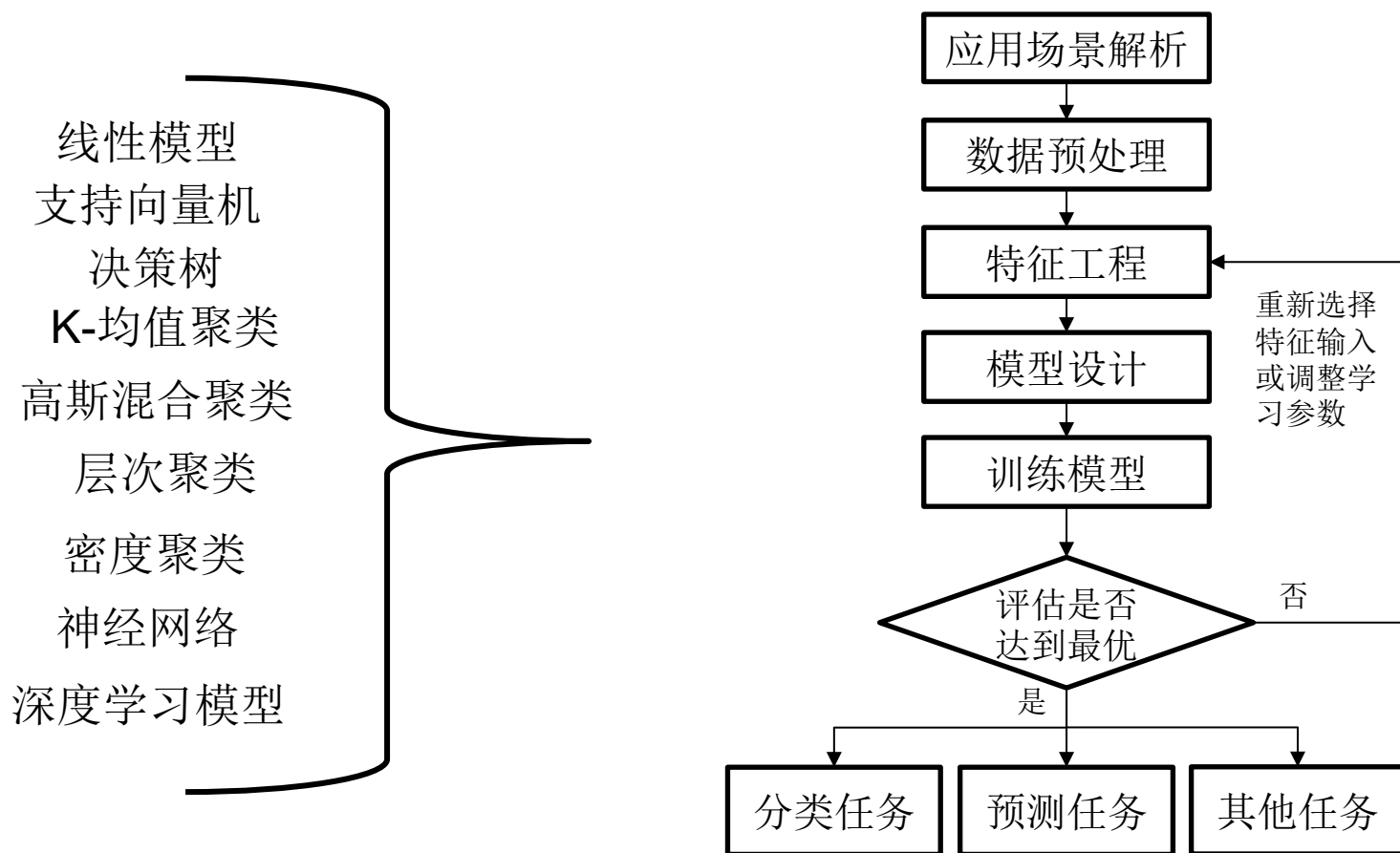
□ 降维问题

在大数据分析问题中，往往因为特征数量过多，导致了特征空间的维度过高，从而耗费大量存储空间，而且严重降低了机器学习算法的计算效率，因此在尽量多地保留数据所携带的重要信息的基础上对高维特征空间做低维近似，这是降维问题的核心目标。



3 机器学习的基本流程

在交通大数据问题中，机器学习的流程可以分为7个步骤，分别是应用场景解析、数据预处理、特征工程、设计模型、模型训练、模型评估、结果输出。他们之间的关系可以由下面的流程图来表示



4 机器学习模型的评估与选择

- 误差与损失函数
- 分类问题的性能度量
- 欠拟合与过拟合



误差与损失函数

□ 误差

对于数据集 (x, y) ，误差是指模型的输出结果 $\hat{y} = f(x)$ 与真实值 y 之间的差异。误差是评估模型优劣的重要标准之一。

模型在训练集上的误差被称为“训练误差”（training error）或“经验误差”（empirical error）。在验证集上的误差被称为“验证误差”（validation error），测试集上的误差被称为“测试误差”（test error）



误差与损失函数

□ 损失函数

为了合理地量化误差，来度量出模型在数据集上的表现，我们可以构造**衡量误差的函数**，这种函数统称为损失函数（loss function）或代价函数（cost function）。

假设数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ 包含 m 个样本，每个样本包含 d 个特征，即 $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})^T$ ，那么基于机器学习模型 $\hat{y} = f(\mathbf{x})$ ，可以得到第 i 个样本对应的模型输出结果 $\hat{y}_i$ 。我们用符号 $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 来表示损失函数，接下来介绍几类常用的损失函数。



误差与损失函数

□ 损失函数

(1) 0-1损失函数(0-1 Loss Function)

对于分类，为了表征分类结果的准确性，我们可以用0-1函数来直观判断样本的分析结果，需要注意的是**0-1损失函数仅适用于二分类问题，应用范围较小**

$$\mathcal{L}_i^{0-1}(y_i, \hat{y}_i) = \begin{cases} 1 & y_i \neq \hat{y}_i \\ 0 & y_i = \hat{y}_i \end{cases}$$

$$\mathcal{L}^{0-1}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}_i^{0-1}(y_i, \hat{y}_i)$$



误差与损失函数

□ 损失函数

(2) 均方误差 (Mean Square Error, MSE)

均方误差的定义如下：

$$\mathcal{L}^{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

均方误差通过平方相加的形式避免了正负误差的相互抵消。均方误差函数有很多优势（1）没有约束条件；（2）连续可微；（3）目标方程是凸函数。因此均方误差最小化问题可以通过求函数的“驻点”来解决



误差与损失函数

□ 损失函数

(3) 平均绝对误差 (Mean Absolute Error, MAE)

平均绝对误差的定义如下：

$$\mathcal{L}^{MAE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

(4) 平均绝对百分比误差 (Mean Absolute Percentage Error, MAPE)

$$\mathcal{L}^{MAPE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$



试一试

利用python在计算机中随机生成一组训练数据与模型输出结果，计算均方误差，平均绝对误差与平均绝对百分比误差

```
import numpy as np
from sklearn.linear_model import LinearRegression

#随机生成训练数据
X_train = np.random.randint(0, 20, size=[5000, 2])
y_train = np.random.randint(0, 20, 5000)

#线性拟合
linreg = LinearRegression()
linreg.fit(X_train, y_train)

##计算估计值
y_estimate = linreg.predict(X_train)

##计算估计误差
error_sum = 0
for i in range(5000):
    error_sum += (y_train[i]-y_estimate[i])**2
MSE = error_sum/5000
print("MSE is", MSE)
```

例：计算线性回归
的均方误差



(5) 交叉熵损失函数 (Cross Entropy Loss Function)

交叉熵 (cross entropy) 是由信息熵 (information entropy) 概念衍生出来的一个概念。对于一个分类问题, 我们用 $\bar{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ 表示测试集的数据其输出空间 y 的可能取值包括 $k = 1, 2, 3, \dots, |y|$, 如果第 k 类样本所占比例为 p_k , 那么样本标签在集合 $\bar{D}$ 中的信息熵定义为:

$$Ent(\bar{D}) = - \sum_{k=1}^{|y|} p_k \log_2(p_k)$$

信息熵是**随机变量不确定的度量**, 是对所有可能发生的事件产生的信息量的期望。而**交叉熵常被用来度量同一个随机变量 x 的两个概率分布 p 和 q 之间的差异**。在测试集 $\bar{D}$ 中我们可以得到模型的预测值 $\hat{y} = f(x)$, 用 q_k 来表示 $\hat{y}$ 的第 k 类所占的比例, 那么真实值 y 和预测值 $\hat{y}$ 的交叉熵被定义为:

$$H(p, q) = - \sum_{k=1}^{|y|} p_k \log_2(q_k)$$

交叉熵的值越小就表明两个分布越接近, 也就是说分类效果越好。



试一试

在计算机中随机生成训练数据，训练逻辑回归模型，计算逻辑回归的交叉熵损失函数值

```
from sklearn.linear_model import LogisticRegression
from collections import Counter
##生成随机训练数据
X_train = np.random.randint(0, 20, size=[50000, 2]) #生成在(0, 1)均匀分布的样本值
y_train = np.random.randint(0, 3, 50000) #生成只含0, 1, 2的随机标签

##训练逻辑回归模型
lr = LogisticRegression(C=1000.0, random_state=0)
lr.fit(X_train, y_train)

##根据训练模型预测
y_estimate = lr.predict(X_train)

##统计不同类别比例
n_estimate = Counter(y_estimate)
n_train = Counter(y_train)
key = list(n_estimate.keys())

##计算交叉熵损失函数
H = 0
for i in range(len(key)):
    k_i = key[i]
    p_i = n_train[k_i]/50000
    q_i = n_estimate[k_i]/50000
    H = H - p_i * np.log2(q_i)

##输出结果
print("交叉熵损失函数值为", H)
```



分类问题的性能度量

我们可以用**准确率 (accuracy)**、**精确率 (precision)**、**召回率 (recall)** 和 **F1-score**来度量分类的准确性。

以二分类问题为例，模型的分类结果与真实情况相比会出现四种情况：真正类 (true positive, TP)，即将正类预测为正类的情况、假正类 (false positive, FP)，即将负类预测为正类的情况、假负类 (false negative, FN)，即将正类预测为负类的情况、真负类 (true negative, TN)，即将负类预测为负类的情况

TP	FN
FP	TN



分类问题的性能度量

(1) 准确率

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}$$

需要注意的是，准确率虽然能够判断整体正确率，但在**样本不均衡**的情况下，并不能作为很好的指标来衡量结果。比如在样本集中，正样本有 90 个，负样本有 10 个，该情况下，我们甚至只需要将全部样本预测为正样本，就能得到 90% 的准确率，显然这是完全没有意义的

(2) 精确率

精确率是**模型预测为正的样本**中，预测正确比例

$$Precision = \frac{TP}{TP + FP}$$

(3) 召回率

召回率是**所有正类样本**被识别正确的比例

$$Recall = \frac{TP}{TP + FN}$$



例子:

如果我们利用网约车（浮动车）数据获取了某个目标路段上1000个时间片段的路况样本，其中只有10个样本的真实标签是“拥堵”（**正类**），其他990个都是“通畅”（**负类**）。使用一个训练好的分类器对这些数据进行预测，最终分类器预测出8个“拥堵”样本。在8个拥堵样本中，有5个是真正拥堵的

试计算这个分类器的准确率、精确率以及召回率

准确率:
$$Acc = \frac{TP + TN}{TP + TN + FP + FN} = \frac{5 + 987}{5 + 987 + 3 + 5} = 99.2\%$$

精确率:
$$Precision = \frac{TP}{TP + FP} = \frac{5}{5 + 3} = 62.5\%$$

召回率:
$$Recall = \frac{TP}{TP + FN} = \frac{5}{5 + 5} = 50\%$$



分类问题的性能度量

以检索系统为例，在检索系统中，如果希望提高**召回率**，即希望更多的相关文档被检索到，就要**放宽**“检索策略”，便会在检索中伴随出现一些不相关的结果，从而影响到精确率。如果希望提高**精确率**，即希望去除检索结果中的不相关文档时，就需要**严格**“检索策略”，便会使一些相关文档不能被检索到，从而影响到召回率。

在设计交通拥堵识别算法或交通事故风险点判别算法时，什么**性能度量指标**需要优先考虑？



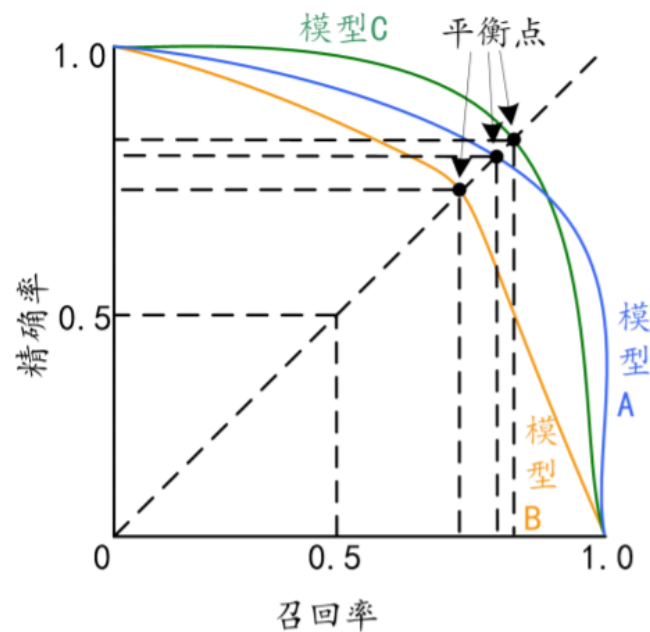
分类问题的性能度量

实际情况下，精确率和召回率往往是此消彼长、无法同时改进的。我们可以通过P-R图、ROC图或F1分数来对学习器进行选择

P-R图 (阅读内容)

P-R图是以召回率为横坐标，精确率为纵坐标，调整分类问题的阈值确定不同阈值下的召回率与精确率，将散点绘制成的曲线。

在P-R图中，如果一个模型的曲线被另一个完全包围，那么前者模型的性能差于后者，如右图中B模型性能比A模型要差，但是如果两个模型的P-R曲线有交叉，就需要在特定的精确率或召回率下来比较，一种常用的方式是找到精确率和召回率的平衡点



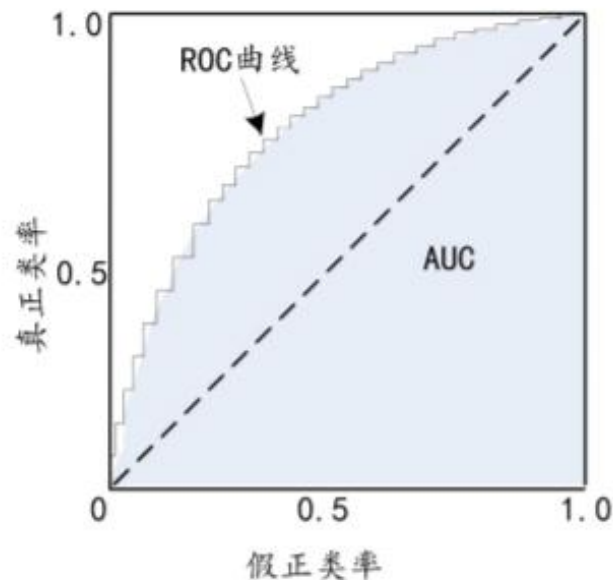
分类问题的性能度量

ROC图 (阅读内容)

与P-R图不同的是，ROC曲线的纵轴是“真正类率”（true positive rate, TPR），横轴是“假正类率”（false positive rate, FPR）

$$TPR = \frac{TP}{TP + FN} \quad FPR = \frac{FP}{FP + TN}$$

与P-R图类似，如果一个模型的ROC曲线被另一个完全包围，那么前者模型的性能差于后者，如果两个模型的P-R曲线有交叉，一般难以断言两个模型孰优孰劣，一种较为合理的判断是比较ROC曲线下的面积就需要在特定的精确率或召回率下来比较，一种常用的方式是比较ROC曲线下的面积，即AUC（area under curve）。



分类问题的性能度量

F1分数 (F1-score)

F1分数是另一种适用于分类问题的性能度量指标，它可以兼顾精确率和召回率。F1分数本质上是精确率和召回率的调和平均值

$$\frac{2}{F_1} = \frac{1}{Precision} + \frac{1}{Recall}$$

化简可以得到：

$$F_1 = \frac{2TP}{2TP + FP + FN}$$



试一试

Python的sklearn模块中自带precision_recall_curve函数。precision_recall_curve函数能快速提供绘制**二分类学习器**P-R曲线所需的精确率和召回率，尝试随机生成样本标签与置信度，调用该函数绘制P-R曲线

➤ 随机生成样本标签与样本置信度

```
import numpy as np
from sklearn.metrics import precision_recall_curve as prc
def precision_recall():
    y_true = np.random.randint(0, 2, 50) #随机生成 50 个样本的标签,取值 0 或 1
    y_scores = np.random.uniform(0, 1, 50) #随机生成每个样本的置信度
    precision, recall, thresholds = prc(y_true, y_scores) #调用precision_recall_curve
    return y_true, y_scores, precision, recall, thresholds
```

➤ 绘图

```
import matplotlib.pyplot as plt
y_true, y_scores, precision, recall, thresholds = precision_recall()
plt.plot(recall, precision)
```



思考P-R图与ROC图有何关系？ **(阅读教材书)**

ROC图是否适合评估正负样本不均衡情况下的分类器性能？

- Davis, J., & Goadrich, M. (2006, June). The relationship between Precision-Recall and ROC curves. In Proceedings of the 23rd international conference on Machine learning (pp. 233-240).

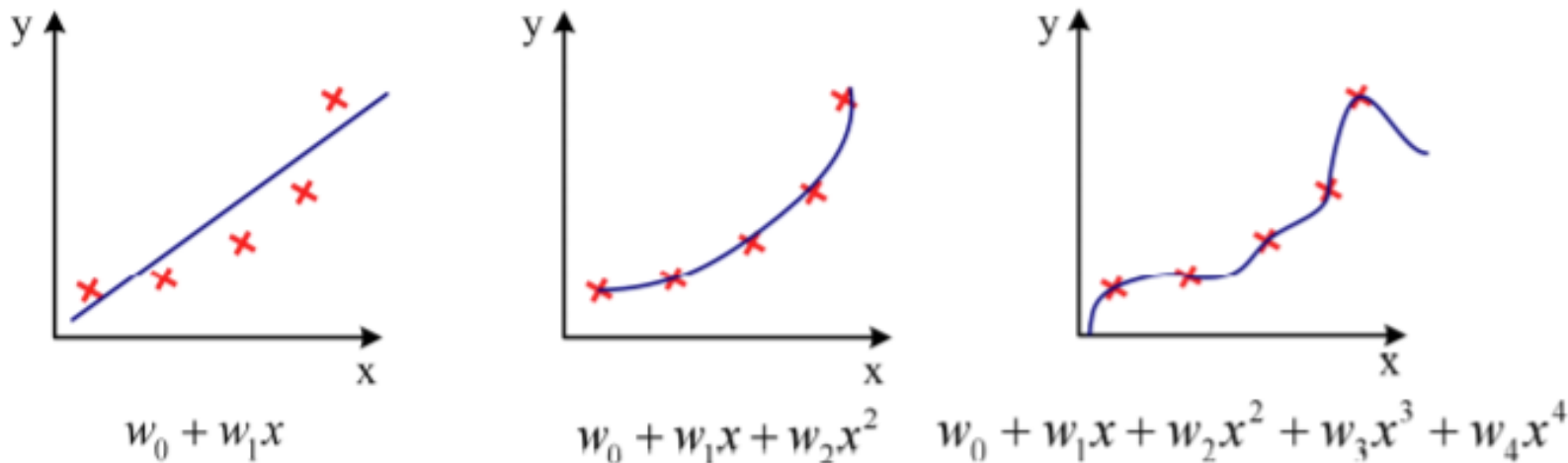


欠拟合与过拟合

在机器学习的模型训练中，有两类常见的问题：欠拟合（underfitting）和过拟合（overfitting）。

- 欠拟合是指模型的拟合程度不高，导致训练误差大；
- 过拟合是指模型的泛化能力差，也就是模型虽然训练误差小，但在新数据上的表现差（也就是泛化误差大）；

下图显示了一组训练数据，欠拟合、合理拟合和过拟合的三种情况



一般而言，欠拟合更容易克服，往往可以通过扩大样本量、提高模型特征数来有效克服。例如在决策树学习中扩展分支，在神经网络学习中增加训练轮数等

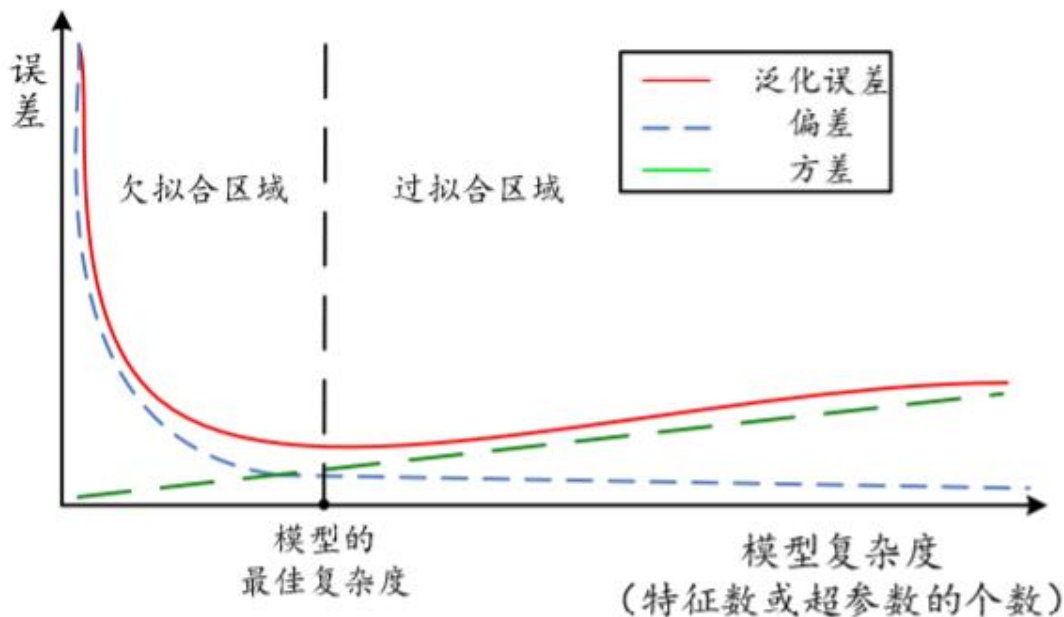
欠拟合与过拟合：方差与偏差

在机器学习中，偏差和方差是常用于描述模型的拟合程度。（高）偏差是指欠拟合，（高）方差是指过拟合。

- **偏差**反映了模型输出值 $\hat{y}$ 和真实值 y 之间的差异程度，即模型的**拟合能力**。
- **方差**反映了模型在不同数据集上的性能稳定性，即数据扰动所带来的影响。

出现高偏差时，训练误差和泛化误差都高；而出现高方差时，训练误差低，泛化误差高

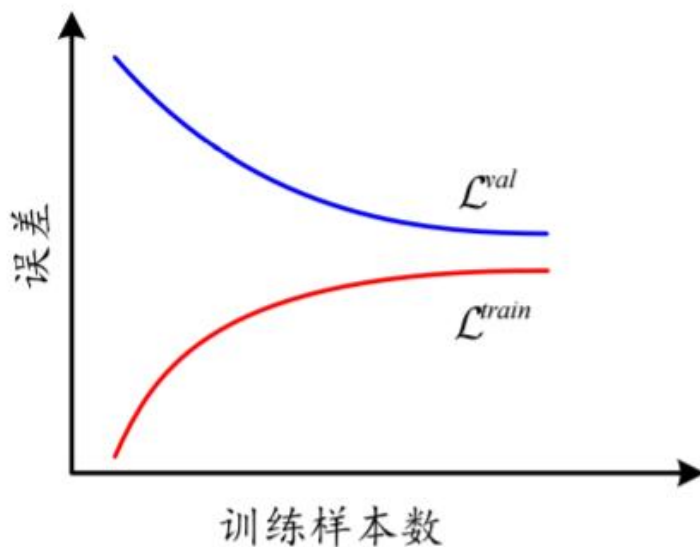
下图画出了随着模型复杂度的增加，偏差、方差和泛化误差的变化情况，我们可以利用这一曲线来选择最佳的模型复杂度



欠拟合与过拟合：学习曲线

（在多维情况下，无法直观地看出来拟合情况）模型的欠拟合和过拟合可以通过“学习曲线”来进行判断。学习曲线是将训练误差和验证误差作为训练样本数的函数绘制的曲线。横轴为训练样本数，纵轴为误差。

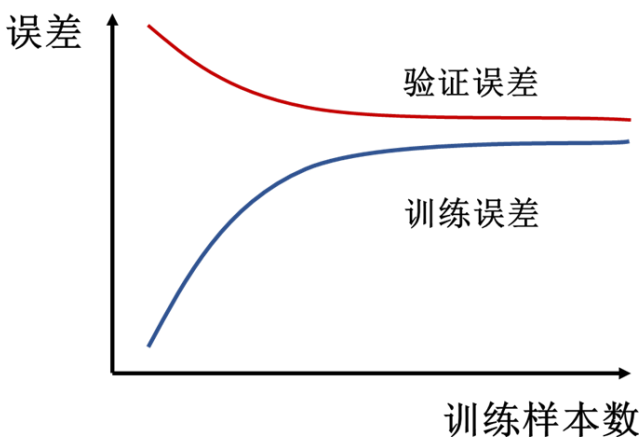
学习曲线的具体画法是：（在相同的模型复杂程度下）把我们拥有的总数为 M 个的训练样本，从第一个样本开始逐步地拿出前 m 个样本，训练得到模型 f_m ，计算这 m 个样本的误差 $\mathcal{L}_m^{train}$ 。利用验证集数据根据模型 f_m 计算出验证误差 $\mathcal{L}_m^{val}$ 。逐步增大 m 值，将一系列训练误差和验证误差的散点连接起来，可以得到学习曲线



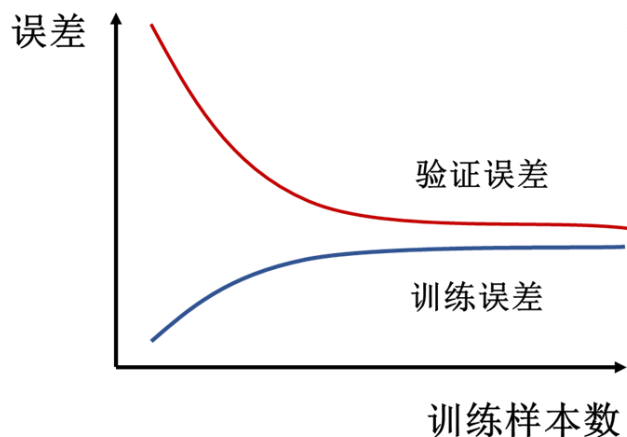
欠拟合与过拟合

模型欠拟合和过拟合情况下的学习曲线是不同的。

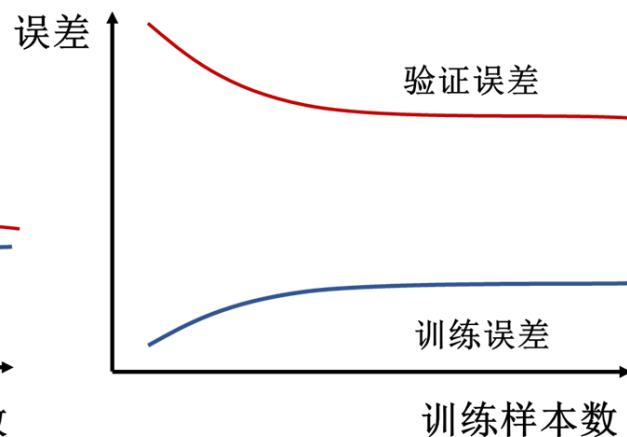
- 在欠拟合情况下，即使训练样本数增加，验证误差和训练误差都比较大
- 在过拟合情况下，随着训练样本数增加，训练误差较小，而验证误差较大



欠拟合



合理拟合



过拟合



试一试

采用随机样本绘制学习曲线，查阅Learning_curve官方文档，了解各个参数含义，调整参数绘制学习曲线，根据学习曲线判断该模型为欠拟合，过拟合还是合理拟合

➤ 随机样本生成：

```
X = np.random.randint(0, 20, size=[50000, 2])  #生成在(0, 1)均匀分布的样本值  
y = np.random.randint(0, 3, 50000)  #生成只含0, 1, 2的随机标签
```

➤ 分类器设置（这里采用对数几率回归模型）：

```
from sklearn.linear_model import LogisticRegression  
estimator = LogisticRegression()
```

➤ Sklearn学习曲线绘制函数：

```
train_sizes, train_scores, test_scores = learning_curve(  
    estimator, X, y, cv=None, n_jobs=1, train_sizes=np.linspace(0.1, 1.0, 6))
```



5 改善模型性能的方法

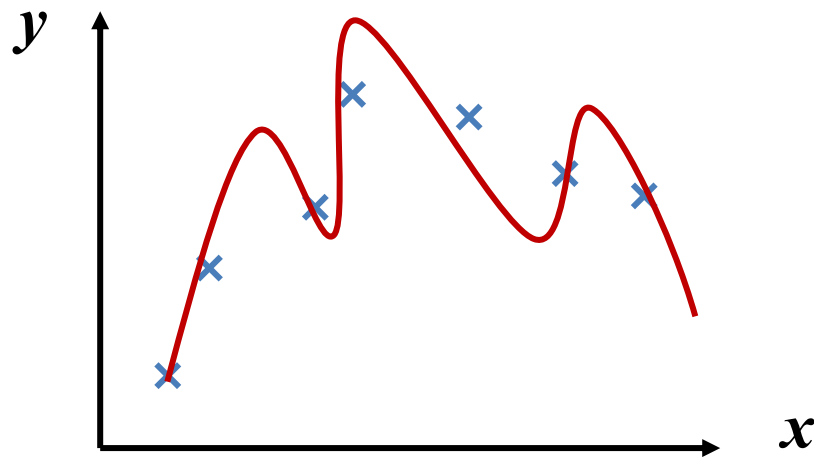
□正则化

□交叉验证



正则化

考虑如下散点的拟合问题，我们以4次多项式拟合散点，发现出现了较为严重的过拟合，如何对这一过拟合问题进行修正？



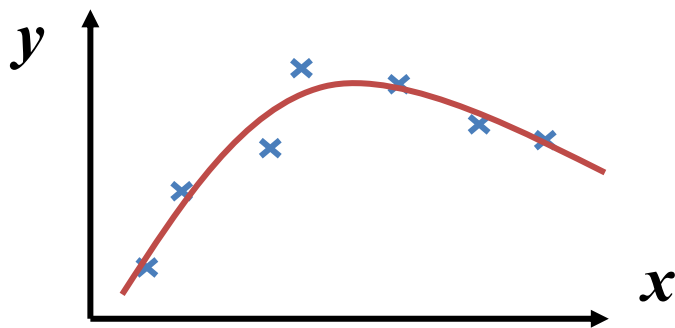
$$y(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

损失函数:
$$f(\boldsymbol{\theta}) = \sum_{i=1}^n (y_i - y(x_i))^2$$

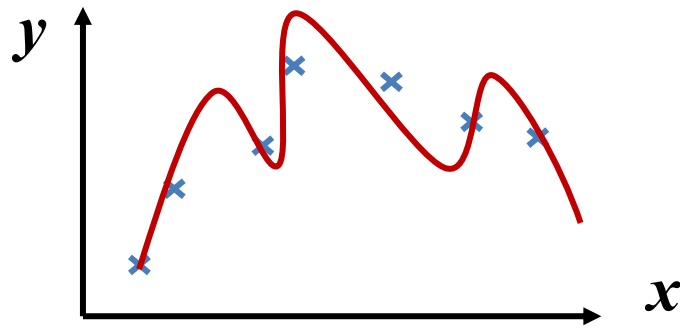


正则化

倘若我们已知二次多项式能够取得更好的拟合效果，但又希望保留四次多项式的拟合形式，我们可以尽量减少3次项系数 θ_3 与4次项系数 θ_4 的值，使之与二次多项式更为接近



$$y = \theta_0 + \theta_1 x + \theta_2 x^2$$



$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

倘若我们不知道何种拟合形式更好？我们该如何对过拟合进行修正呢？



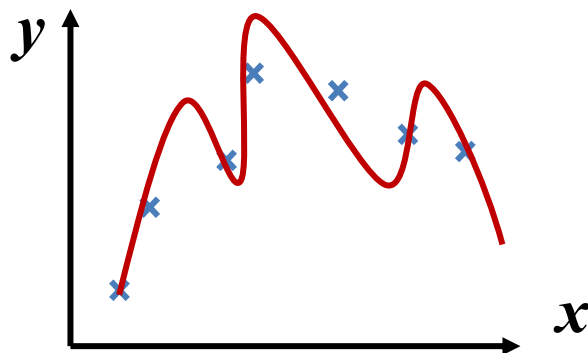
$$g(\theta) = f(\theta) + 100|\theta_3| + 100|\theta_4|$$

通过调整损失函数减小 θ_3 与 θ_4 的值



正则化

倘若我们不知道何种拟合形式更好？我们该如何对过拟合进行修正呢？



参考减小 θ_3 与 θ_4 的思路，我们可以通过修正损失函数的形式，减小拟合函数中每一项的系数（ θ_0 除外），达到简化模型的目的（对过拟合进行修正）

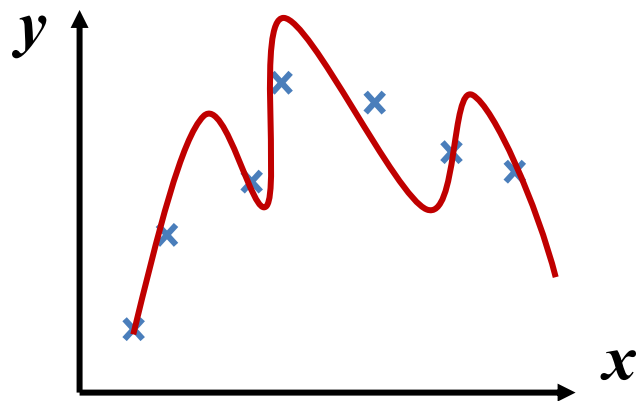
$$f(\boldsymbol{\theta}) = \sum_{i=1}^n (y_i - y(x_i))^2 \quad \Rightarrow \quad g(\boldsymbol{\theta}) = f(\boldsymbol{\theta}) + \sum_{i=1}^4 \lambda |\theta_i|$$

可以直观地理解，如果拟合函数中 θ_i $i \in \{1, 2, 3, 4\}$ 前的系数为无穷大，那么 θ_i $i \in \{1, 2, 3, 4\}$ 将趋于0，也就是说，此时， $y = \theta_0$ ，这是最简化的模型



正则化

倘若我们不知道何种拟合形式更好？我们该如何对过拟合进行修正呢？



$$g(\boldsymbol{\theta}) = \sum_{i=1}^n (y_i - y(x_i))^2 + \sum_{i=1}^4 \lambda |\theta_i|$$

这种在损失函数基础上添加系数绝对值之和以减轻模型训练过拟合的处理方法叫做L1正则化，除了L1正则化处理，还有一种常见的处理思路，即增添为系数平方和，这种处理方法叫做L2正则化

L2正则化:

$$g(\boldsymbol{\theta}) = \sum_{i=1}^n (y_i - y(x_i))^2 + \sum_{i=1}^4 \lambda \theta_i^2$$

请注意，增添在损失函数上的项前的系数值 λ 需要根据实际情况确定



正则化 (阅读内容)

当模型的复杂度过高时，会导致过拟合，我们可以用正则化的方法来限制模型的复杂度。正则化的思想是**在模型的损失函数中添加一个惩罚项，从而使一部分系数的最优取值尽可能小，也就是弱化这些参数的作用。**

然而在实践中模型的输入维度往往很高，难以人工选取参数来进行惩罚，因此最常用的一种正则化方法是直接在损失函数中添加一个正则化项来限制所有参数。我们定义**结构风险** (structural risk)：

$$R_{str}(f) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(y_i, f(x_i)) + \lambda J(f)$$

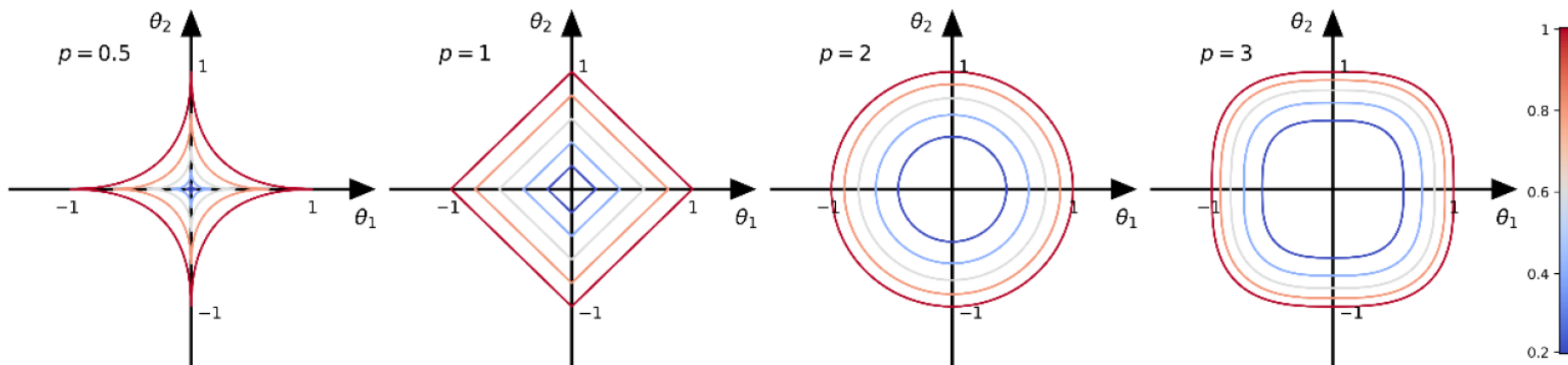
其中 $J(f)$ 表示模型的复杂度， λ 大于0表示对复杂模型的惩罚力度。正则化就是，用结构风险最小化 (structural risk minimization, SRM) 来替代损失函数作为模型优化的目标方程

模型复杂度最常用的度量方法是参数向量的 L_p 范数，对于向量 x ，其 L_p **范数**由下式给出：

$$L_p(x) = \|x\|_p = \left(\sum_{i=1}^m |x_i|^p \right)^{\frac{1}{p}}$$



正则化 (阅读内容)

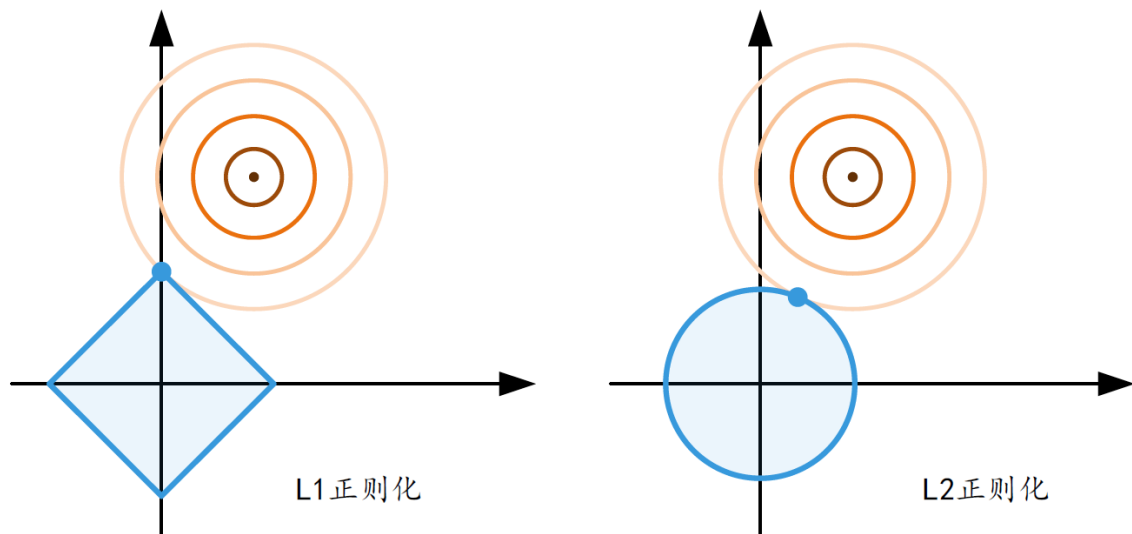


两个参数 (θ_1, θ_2) , 不同 p 值下模型复杂度等值线

- 在正则化中，常用的 p 值为1和2。分别对应于L1正则化和L2正则化。直观上理解，L1 正则化就是在损失函数中添加模型全部参数绝对值的和，而 L2 正则化则是添加模型全部参数的平方和。

正则化 (阅读内容)

以二维参数向量的情况为例，多个同心圆为MSE损失函数相对于不同模型参数的等值线，其圆心为无正则项条件下模型的最优参数取值；左下角的菱形和圆形分别为L1正则项和L2正则项对应的等值线，其与同心圆的切点为使用相应正则项后的模型最优参数取值。在MSE相同的情况下，使L1正则项最小的参数组合容易出现在**坐标轴上**，此时有一个参数为0；而使L2正则项最小的参数最小的参数则不然。由于稀疏解中许多参数为0，因此L1正则化在减小模型复杂度的同时，还能够起到**特征选择**的作用。



L1正则化和L2正则化对解的影响

正则化 (阅读内容)

- 除了模型复杂度的表示，正则项系数的大小也会对模型的拟合效果产生影响。相比不添加正则项时学习到的函数，使用恰当的正则化可以明显缓解过拟合问题。但是，过大的正则化系数 λ ，则会导致模型的全部参数都趋近于 0，造成严重的欠拟合问题。需要注意的是，正则化系数 λ 的设置没有确定的规则。通常可从较小的 λ 值开始实验，以一定间隔不断增减尝试，确定合适的数量级后再进行微调



试一试

自行生成训练数据与测试数据，采用多项式函数拟合数据，分别观察不进行正则化、L1正则化、L2正则化时回归曲线的差异

➤ 生成训练数据与测试数据

```
##生成训练数据
x = np.arange(-3, 3, 0.5)
X_train = x.reshape(-1, 1)
y_train = 0.5 * x**2 + x + 2 + np.random.normal(0, 2, len(x))
##测试数据
x_test = np.arange(-3, 3, 0.01)
```

➤ 定义拟合多项式，根据训练数据转换特征

```
##degree为拟合多项式的最高次
Polynomial_featurizer = PolynomialFeatures(degree)
X_train_poly = Polynomial_featurizer.fit_transform(X_train)
x_test_poly = Polynomial_featurizer.fit_transform(x_test.reshape(x_test.shape[0], 1))
```



试一试

自行生成训练数据与测试数据，采用多项式函数拟合数据，观察不进行正则化、L1正则化、L2正则化下回归曲线的差异

➤ 进行拟合

```
##不加惩罚项
lr = LinearRegression()
lr.fit(X_train_poly, y_train)
##L1惩罚项
la = Lasso(lamda1)
la.fit(X_train_poly, y_train)
##L2惩罚项
rd = Ridge(lamda2)
rd.fit(X_train_poly, y_train)
```

➤ 绘图

```
plt.plot(X_train, y_train, 'k.')
plt.plot(x_test, lr.predict(x_test_poly), 'r-', label = "No regularization")
plt.plot(x_test, rd.predict(x_test_poly), 'b-', label = "L2 regularization")
plt.plot(x_test, la.predict(x_test_poly), 'g-', label = "L1 regularization")
plt.legend()
```



交叉验证

在机器学习模型的选择和训练，数据集常被划分为**训练集 验证集和测试集**三个部分。

- 训练集用于模型的训练，获得模型参数；
- 验证集用于优化模型的超参数，也即选择不同模型；
- 测试集用于对模型最终效果的评估。然而实际应用中数据量很可能是有限的，验证集不具备足够的数据来进行模型验证，筛选最优模型。此时需要进行交叉验证

交叉验证的主要思想是将数据集分为训练集和测试集后，再将训练集切分为若干份子数据集，再将这些子数据集组合为训练集和验证集，在此基础上进行多次训练和验证。交叉验证具体可分为两类：S折交叉验证（S-fold cross validation）和留一交叉验证（leave-one cross validation）

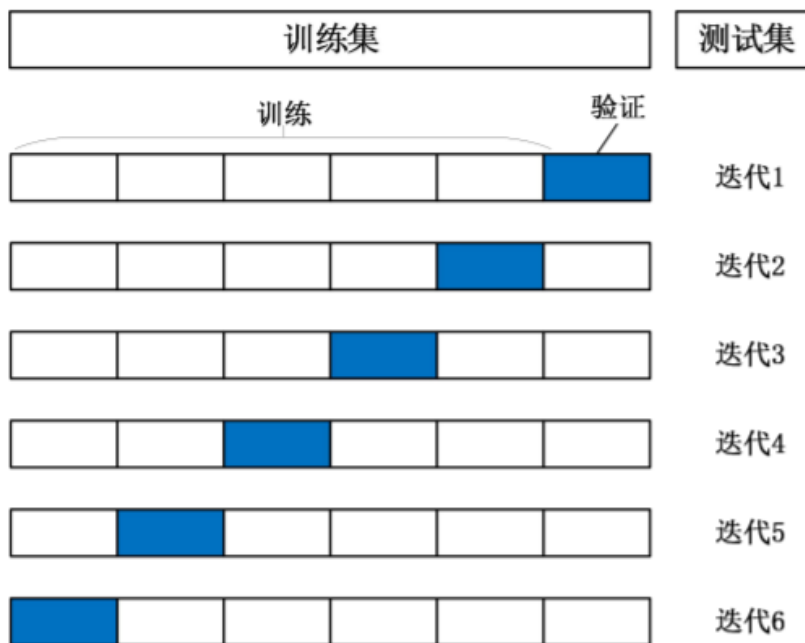


交叉验证

□ S折交叉验证 (S-fold cross validation)

S折交叉验证则将训练集分割成**大小相同的 S 组数据集**，我们先使用2~ S组数据集作为新的训练集进行训练，后用第一组数据集进行验证；接下来使用第二组数据集作为验证数据，而剩余数据集用于训练，依次往复，直至遍历完所有数据集。最后能获得 S 组评价指标（误差），**取均值**后即可获得当前超参数下的模型平均性能。下图展示了6折交叉验证的数据集划分方法。

在S折交叉验证中，每一个样本数据都即被用作训练数据，也被用作测试数据。一定程度上可以避免训练的过**拟合与欠拟合**情况的发生



交叉验证

□ 留一交叉验证 (leave-one cross validation)

留一交叉验证实际上是S折交叉验证的一种特殊情况。在数据量非常缺乏的情况下，另 $S=N$ ， N 为数据集的容量，每次只用一个样本来进行验证，这种交叉验证方法被称为留一交叉验证法



试一试

结合上一练习中的对数几率回归模型，编写函数。

功能：给定训练样本，训练模型，采用S折交叉验证方法评估模型，要求函数可调整交叉验证折数

➤ **计算损失函数
(这里采用交叉熵损失函数)**

```
def Cross_entropy_loss_function(n_estimate, n_train, N):  
    H = 0  
    for i in range(len(n_estimate)):  
        p_i = n_train[i]/N  
        q_i = n_estimate[i]/N  
        H = H - p_i * np.log2(q_i)  
    return H
```

➤ **数组拼接函数**

```
def Array_Joint(A, B):  
    Result = []  
    for a in A:  
        Result.append(a)  
    for b in B:  
        Result.append(b)  
    return np.array(Result)
```


试一试

➤ S折交叉验证函数

```
def S_fold_cross_validation(X_train, Y_train, S):  
    lr = LogisticRegression(C=1000.0, random_state=0)  
    sample_num = int(len(X_train)/S)  
    H = []  
    for i in range(S-1):  
        x_test = X_train[i*sample_num:(i+1)*sample_num]  
        y_test = Y_train[i*sample_num:(i+1)*sample_num]  
  
        x_train = Array_Joint(X_train[:i*sample_num] , X_train[(i+1)*sample_num:])  
        y_train = Array_Joint(Y_train[:i*sample_num] , Y_train[(i+1)*sample_num:])  
  
        ##训练模型  
        lr.fit(x_train, y_train)  
        ##根据训练模型预测  
        y_estimate = lr.predict(x_test)  
  
        ##统计不同类别比例, 计算损失函数  
        n_estimate = Counter(y_estimate)  
        n_test = Counter(y_test)  
        N_test = len(x_test)  
        h = Cross_entropy_loss_function(n_estimate, n_test, N_test)  
        H.append(h)  
  
        ##对最后一折单独考虑, 避免样本数无法整除S的情况  
        x_test = X_train[(S-1)*sample_num:]  
        y_test = Y_train[(S-1)*sample_num:]  
  
        x_train = X_train[: (S-1)*sample_num]  
        y_train = Y_train[: (S-1)*sample_num]  
  
        lr.fit(x_train, y_train)  
        y_estimate = lr.predict(x_test)  
  
        n_estimate = Counter(y_estimate)  
        n_test = Counter(y_test)  
        N_test = len(x_test)  
        h = Cross_entropy_loss_function(n_estimate, n_test, N_test)  
        H.append(h)  
    return np.mean(H)
```

6 机器学习训练中的其他技巧

- 利用数据改善模型性能

- 特征选择技巧



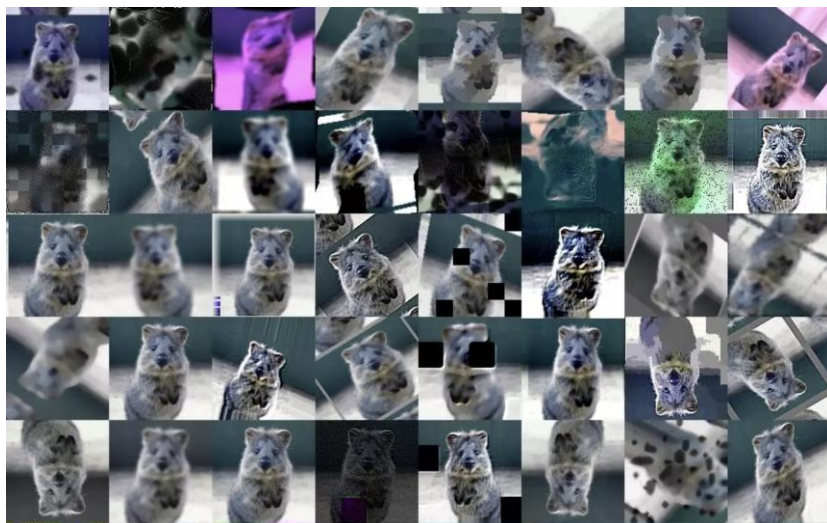
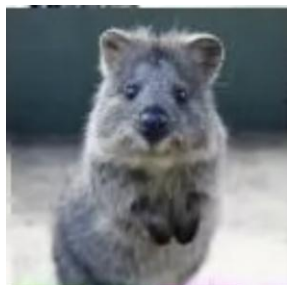
利用数据改善模型性能

从数据预处理的角度，对数据进行扩充、清洗、变换、筛选等工作，是提升机器学习模型性能的方式之一，从数据角度改善模型性能可以从以下方面入手：

(1) 人工扩充数据集

可以通过对有限数据的变换来人工扩充数据集，例如填充或重新排列现有数据；利用概率模型生成新的数据；增加原始数据扰动；使用滑动窗口对数据进行裁切等

例如，图像识别时，对同一张图像，通过图像的平移、旋转、缩放、调节亮度等措施生成大量的新的数据集



利用数据改善模型性能

(2) 数据重采样

重采样包括欠采样和过采样。当我们想要获得一种模型的基础效果，而当前数据集过大，此时便需要进行欠采样，从而提升训练速度。过采样一般用于某类特征的样本过少的情况，例如分类问题中，几类样本数量极度不平衡，便需要对小数量类别样本进行过采样（例如，有放回重复采样重新生成训练样本，同时增大**小样本的被采集权重**），从而使模型学习其特征。

在识别电信诈骗交易时，常遇到的一个问题是，极少量的诈骗类交易记录淹没在海量的正常交易记录中，一般的分类算法会将这些诈骗记录视作噪声，导致算法无法获得令人满意的分类效果，应对这种情况，常采用的方法是过采样，增加诈骗记录占比。

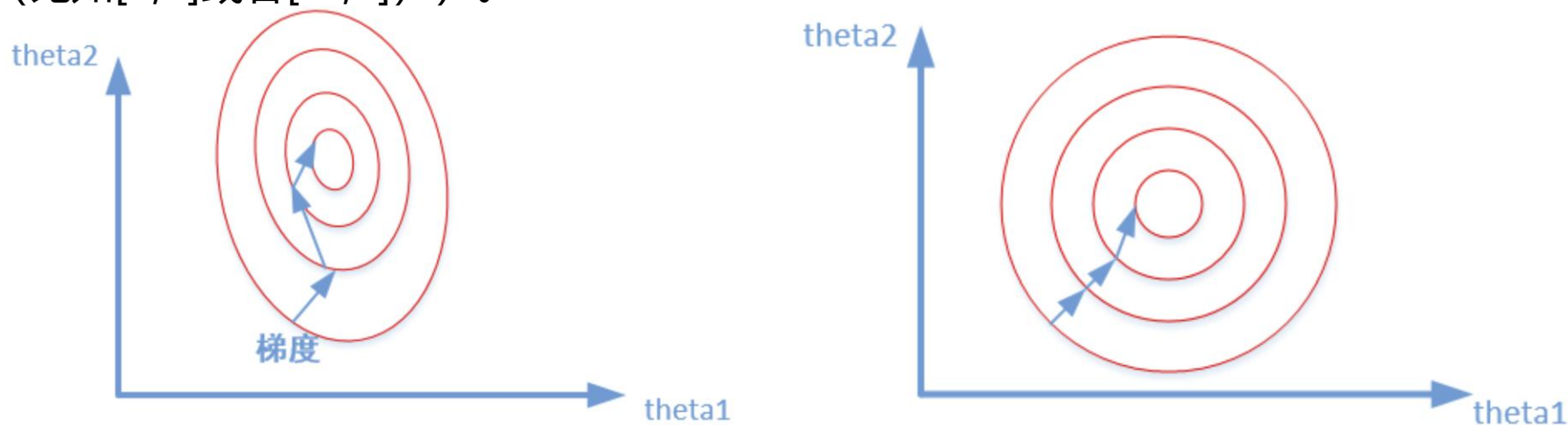


利用数据改善模型性能

(3) 缩放数据 (标准化处理)

模型的输入数据中，不同的属性数值分布范围可能不同，在某些需要计算样本间距离的算法中，可能就会导致数值大的属性对距离计算的影响权重过大，导致训练困难。因此，需要对数据进行缩放，使各属性的数值分布范围一致，常用方法有 0-1 标准化，均值-方差标准化。

最典型**标准化处理**的就是数据的**归一化处理**（使得预处理的数据被限定在一定的范围内（比如 $[0,1]$ 或者 $[-1,1]$ ））。



左图为未进行归一化处理时某一线性回归目标函数等值线；右图为归一化后目标函数等值线

利用数据改善模型性能

- 0-1标准化是指将所有数据缩放到0-1范围内

$$x_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

这一方法的缺点在于，每增加一个新样本，就需要重新计算最大值与最小值

- 均值-方差标准化，可以将数据转换为标准正态分布

$$x_i = \frac{x_i - \mu}{\sigma}$$

其中， μ 为样本总体分布的均值， σ 为样本总体分布的标准差，由于总体均值与方差难以获取，通常采用样本估计均值与标准差进行均值-方差标准化



试一试

对波士顿房价数据集的样本特征进行0-1标准化,

数据集已经导入在“交通大数据课上练习”文件中, 每一条样本数据包含14个特征, 依次为:

- CRIM--城镇人均犯罪率
- ZN - 占地面积超过25,000平方英尺的住宅用地比例。
- INDUS - 每个城镇非零售业务的比例。
- CHAS - Charles River虚拟变量 (如果是河道, 则为1;否则为0)
- NOX - 一氧化氮浓度 (每千万份)
- RM - 每间住宅的平均房间数
- AGE - 1940年以前建造的自住单位比例
- DIS加权距离波士顿的五个就业中心
- RAD - 径向高速公路的可达性指数
- TAX - 每10,000美元的全额物业税率
- PTRATIO - 城镇的学生与教师比例
- B - $1000 (B_k - 0.63)^2$ 其中 B_k 是城镇黑人的比例
- LSTAT - 人口状况下降%
- MEDV - 自有住房的中位数报价, 单位1000美元

请对所有特征进行标准化



利用数据改善模型性能

(4) 转换数据分布

在某些机器学习模型中，往往假定特征服从某种分布（例如正态分布），因此可在预处理阶段，对属性数值分布进行统计性分析，例如，若某个输入属性分布为长尾分布，可对其取对数，使变换后的数值更接近正态分布。

为什么要转换为正态分布？

- 很多模型的应用假设即为，数据分布为正态分布；
- 根据中心极限定理，大量相互独立随机变量的均值经适当标准化后依分布收敛于正态分布，采用正态分布数据训练出的模型具有更好地泛化能力



特征选择技巧

模型特征的选择对于模型的效果有很大的影响，在选择特征时需要仔细考虑，下面介绍几种特征选择思路

- 最直接的方法是直接通过**对特征进行组合**，获得不同的特征子集进行建模，并比较各种特征组合训练得到的模型效果。根据模型效果，选取对应的特征组合。
- 可考虑不训练模型而直接使用一些方法来度量特征组合方案、衡量特征对于输出值的重要程度。在这些度量方法中，**最为常用的是皮尔逊相关系数**。通过计算各输入特征与输出特征的相关系数，选取相关性高的特征。其他的度量方法还包括，利用互信息（mutual information）、卡方检验等来评估特征方案与输出值的相关程度。



特征选择技巧

- 某些模型的内部机制能够计算特征对于输出的重要性，例如以树模型为基础的随机森林模型，通过计算特征的基尼系数、信息增益等，选择特征对数据集进行划分，因此在正式建模前，可先建立一个简单的决策树模型，获得每个特征的重要性指标，进而筛选特征

在机器学习实战中，面临特征选择时，最重要的是从问题本身出发，依据自身对问题的深入理解选取适当的特征，不应迷信特征选择技巧

